

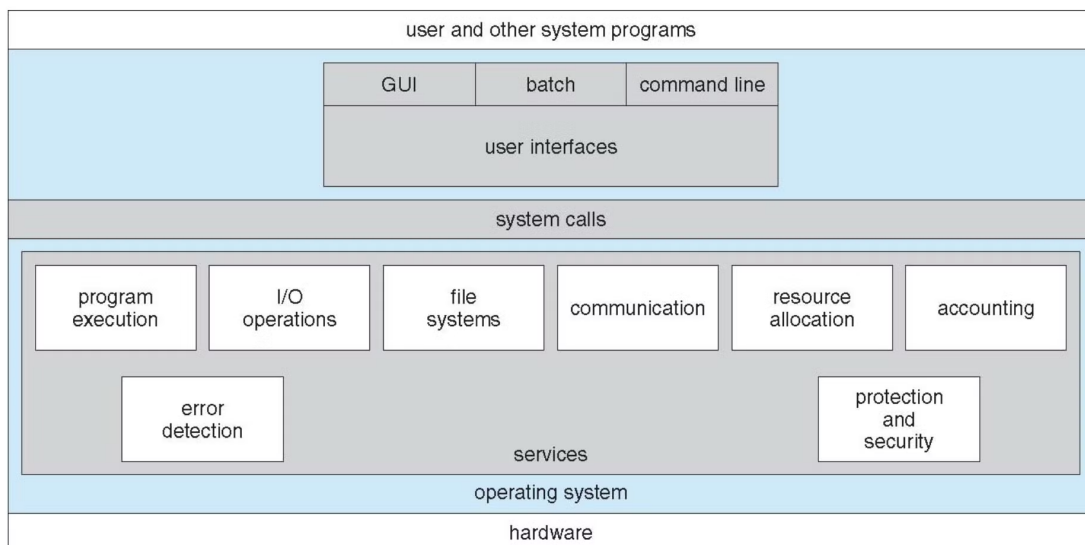


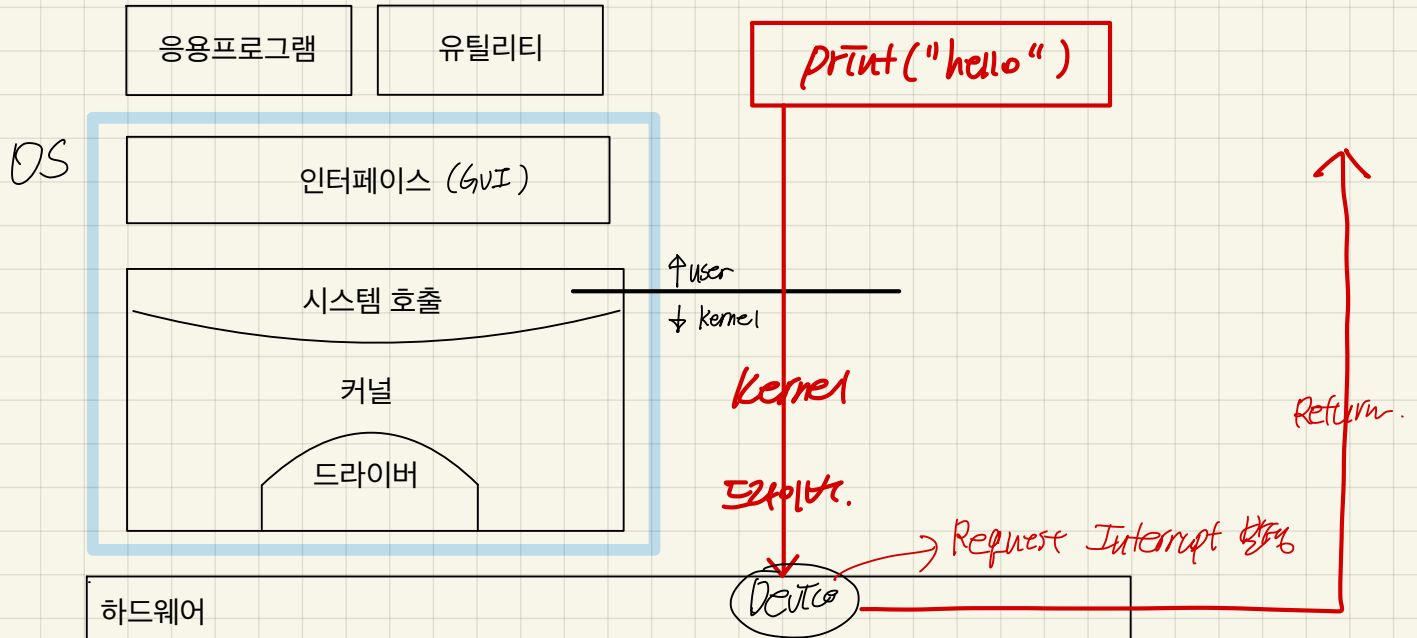
Chap03 요약

1. OS의 역할

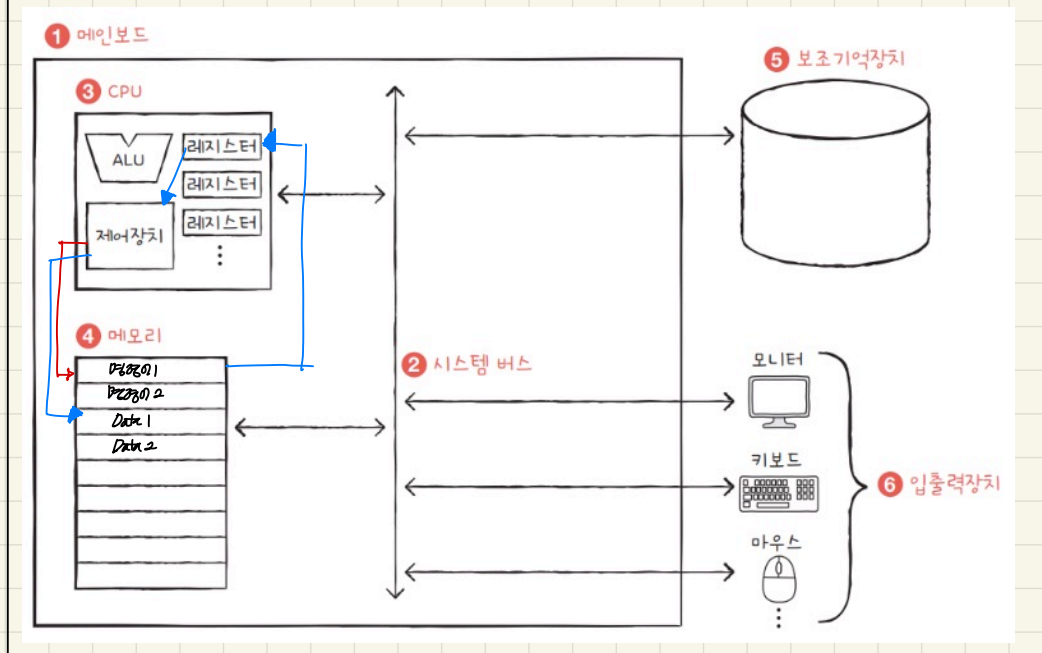
1. CPU 스케줄링과 프로세스 관리
2. 메모리 관리
3. 디스크 파일 관리
4. I/O 디바이스 관리

2. OS 구조 및 동작 방식

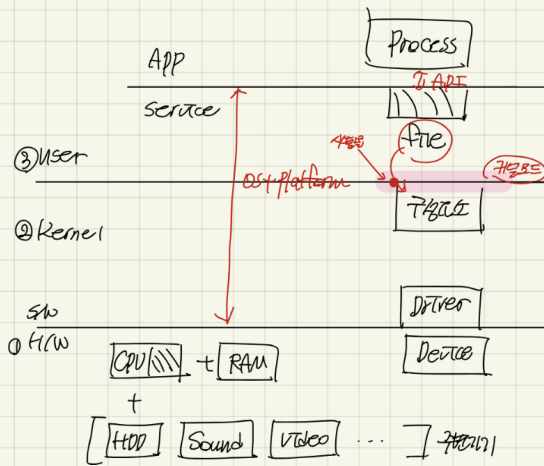




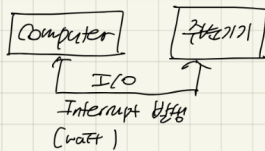
하드웨어



컴퓨터



Interrupt란?



ex) print "hello world"

- ① API를 통해 print 함수를 call
- ② API 내부의 Interface를 통해 접근
- ③ System call을 통해 print 함수에 접근

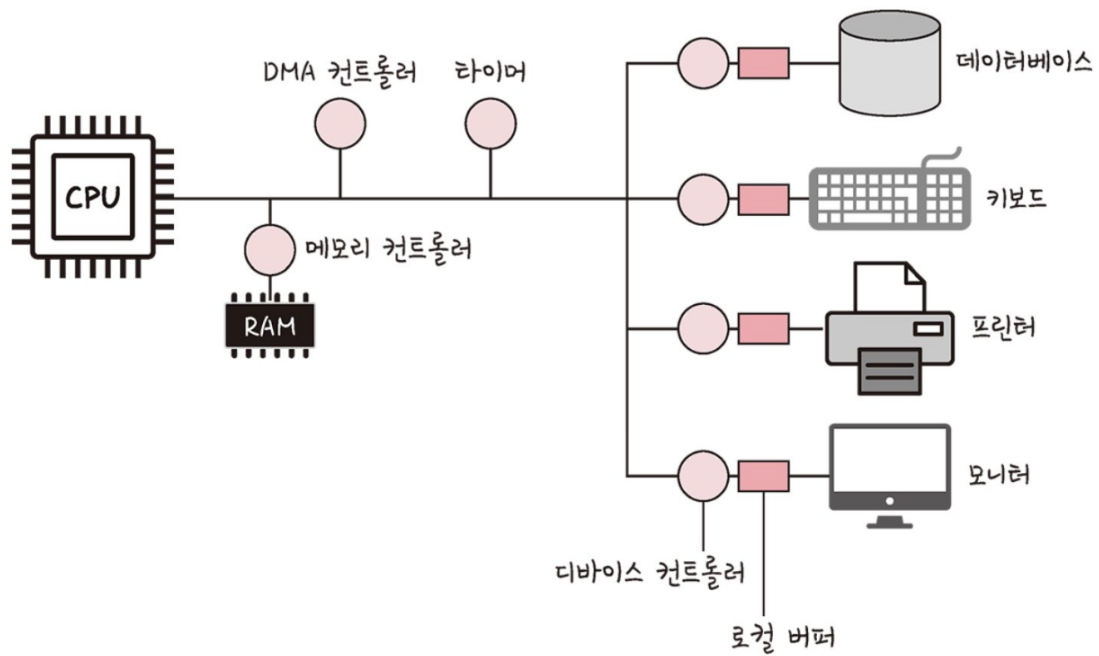
④ Device Driver를 호출하여 처리



* 이때 **호출된 프로세스가 대기 상태**
= blocking I/O

호출된 프로세스 대기, 다른 작업 진행
= unblocking I/O

3. 컴퓨터의 요소

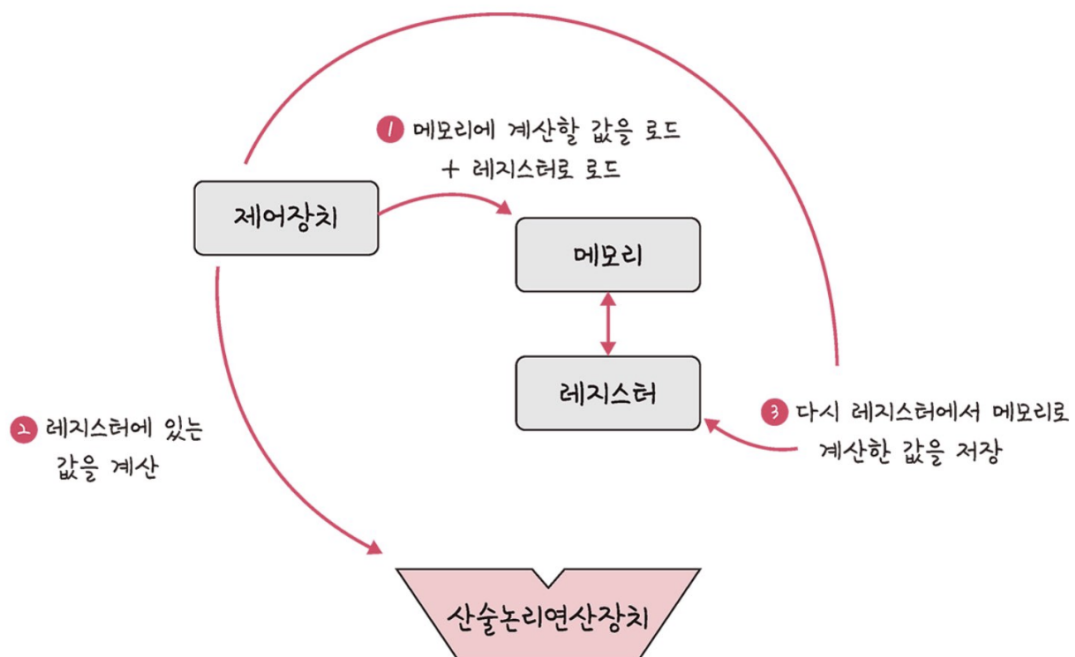


▲ 그림 3-5 컴퓨터의 요소

✓ CPU

= 산술논리연산장치 + 제어장치 + 레지스터

= 인터럽트에 의해 단순히 메모리에 존재하는 명령어 해석



1. 제어장치가 메모리에 계산할 값 로드 , 레지스터에도 로드

2. 제어장치가 레지스터에 있는 값을 계산하라고 산술논리연산장치에 명령
3. 제어장치가 계산된 값을 다시 '레지스터에서 메모리로' 계산한 값을 저장

✅ DMA 컨트롤러

I/O 디바이스가 메모리에 직접 접근할 수 있도록 하는 하드웨어 장치를 뜻함

- CPU 부하를 막해주며 CPU의 일을 부담하는 보조 일꾼
- 하나의 작업을 CPU와 DMA 컨트롤러가 동시에 하는 것을 방지

✅ 디바이스 컨트롤러

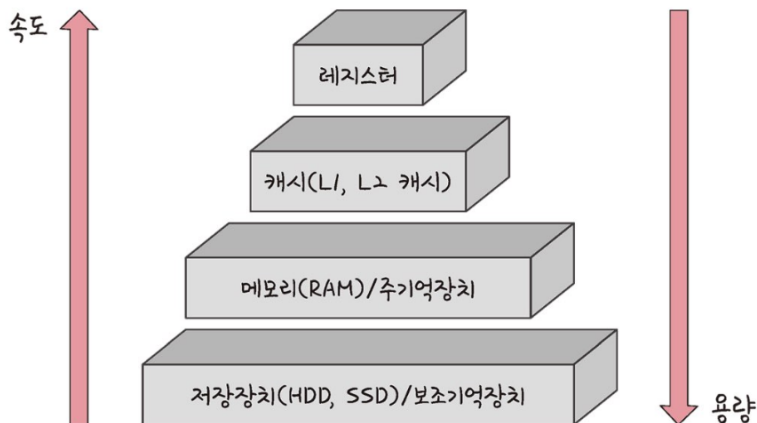
컴퓨터와 연결되어 있는 IO 디바이스들의 작은 CPU를 말함

✅ 로컬 버퍼

데이터를 임시로 저장하기 위한 작은 메모리

4. 메모리 계층

메모리 계층 : 레지스터/캐시/메모리/저장장치



▲ 그림 3-8 메모리 계층

- 레지스터: CPU 안에 있는 작은 메모리/휘발성/
- 캐시 : L1, L2 캐시 지정/휘발성/L3 캐시도 있음/
- 주기억장치: =RAM/휘발성
- 보조기억장치: =HD, SSD/비휘발성/

*RAM은 HD로부터 일정○량의 데이터를 복사해서 임시 저장하고 필요시마다 CPU에 빠르게 전달

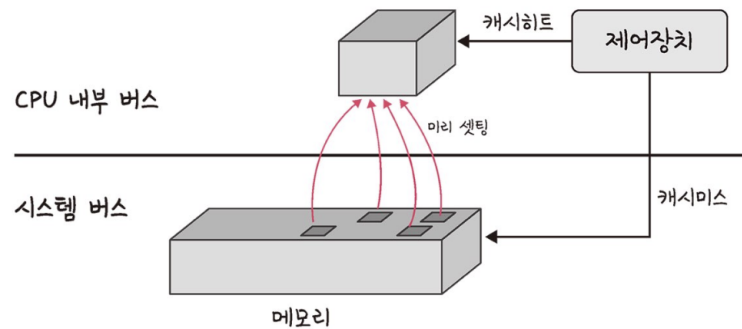
캐시

= 데이터를 미리 복사해 놓은 임시 저장소

= 빠른 장치와 느린 장치에서 속도 차이에 따른 병목 현상을 줄이기 위한 메모리

캐시히트 vs 캐시미스

- 캐시 히트 = 캐시에서 원하는 데이터를 찾았을 경우
- 캐시 미스 = 해당 데이터가 캐시에 없다면 주 메모리로 가서 데이터를 찾아오는 것



▲ 그림 3-9 캐시히트와 캐시미스

캐시매핑

= 캐시가 히트되기 위해 매핑하는 방법

= CPU의 레지스터와 RAM 간에 데이터를 주고받을 때를 기반으로 설명

▼ 표 3-1 캐시매핑 분류

이름	설명
직접 매핑 (directed mapping)	메모리가 1~100이 있고 캐시가 1~10이 있다면 1:1~10, 2:1~20... 이런 식으로 매핑하는 것을 말합니다. 처리가 빠르지만 충돌 발생이 잦습니다.
연관 매핑 (associative mapping)	순서를 일치시키지 않고 관련 있는 캐시와 메모리를 매핑합니다. 충돌이 적지만 모든 블록을 탐색해야 해서 속도가 느립니다.
집합 연관 매핑 (set associative mapping)	직접 매핑과 연관 매핑을 합쳐 놓은 것입니다. 순서는 일치시키지만 집합을 뒤서 저장하며 블록화되어 있기 때문에 검색은 좀 더 효율적입니다. 예를 들어 메모리가 1~100이 있고 캐시가 1~10이 있다면 캐시 1~5에는 1~50의 데이터를 무작위로 저장시키는 것을 말합니다.

웹 브라우저 캐시

■쿠키

- = 만료기한이 있는 **key : value** 저장소
- = 웹 브라우저 닫아도 유지
- = 클라이언트에서만 수정 가능
- = 4KB까지 데이터 저장 가능

[쿠키의 구성 요소]

이름	각각의 쿠키를 구별하는데 사용되는 이름
값	쿠키의 이름과 관련된 값
유효시간	쿠키의 유지시간
도메인	쿠키를 전송할 도메인
경로	쿠키를 전송할 요청 경로

[쿠키의 동작 방식]

1. 클라이언트가 페이지 요청
2. 서버에서 쿠키 생성
3. HTTP 헤더에 쿠키 포함시켜 응답
4. 브라우저가 종료되어도 쿠키 만료 기간있으면 클라이언트에서 보관
5. 같은 요청을 할 경우 HTTP 헤더에 쿠키 함께 보냄
6. 서버에서 쿠키를 읽어 이전 상태 정보를 변경 할 필요가 있을 때 쿠키를 업데이트 하여 변경된 쿠키를 HTTP 헤더에 포함시켜 응답

■로컬 스토리지

→ 쿠키의 단점을 보완하기 위해 html5에서 추가됨

- = 만료기한이 없는 **key : value** 저장소
- = 웹 브라우저 닫아도 유지
- = 클라이언트에서만 수정 가능
- = 5MB 까지 저장 가능
- = 도메인이 다른 경우 공유 불가능

`http://test.com/user`라는 URL에 로컬 스토리지 정보를 저장하였다고 가정을 하였을 경우에 `http://test2.com/user`로 접근하였을 경우 해당 로컬스토리지의 값은 유효하지 않습니다.

[로컬 스토리지의 사용]

| 💡 '로컬 스토리지'에서 사용되는 '메서드'

메서드	설명
setItem (key, value)	키(key)와 값(value)을 기반으로 저장합니다.
getItem (key)	키(key) 값을 기반으로 값(value)을 불러옵니다.
key (index)	인덱스(index) 값을 기반으로 값(value)을 불러옵니다.
removeItem (key)	키(key) 값을 기반으로 해당 로컬 스토리지를 제거합니다.
clear ()	로컬 스토리지를 초기화 합니다.
length	로컬 스토리지에 저장된 데이터 개수를 반환 받습니다.

■ 세션 스토리지

→ 쿠키의 단점을 보완하기 위해 html5에서 추가됨

= 만료기한이 없는 **key : value** 저장소

= 탭 단위로 세션 스토리지 생성

= 탭 닫을 경우 데이터 삭제됨

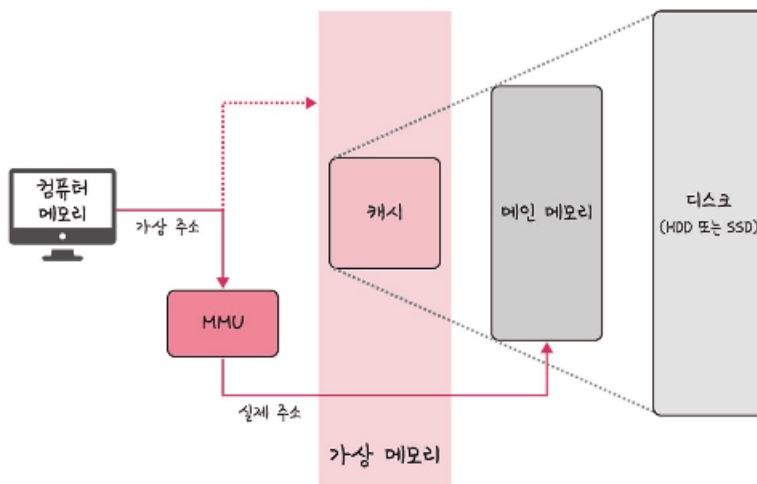
= 5MB까지 저장 가능

5. 메모리 관리

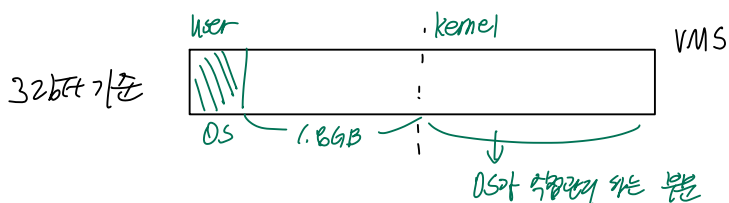
✅ 가상 메모리

메모리 관리 X
관리 / 접근 통제 가능 → 보안성 유

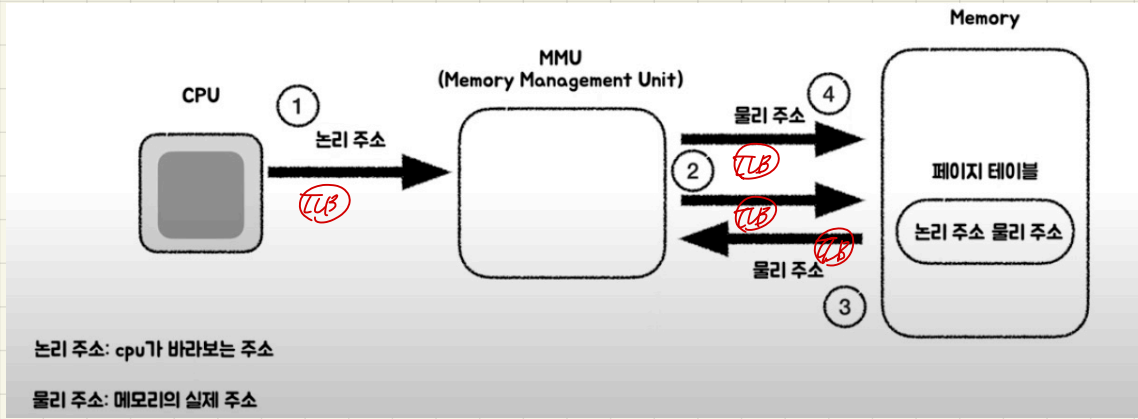
메모리 관리 기법의 하나로 컴퓨터가 실제로 이용 가능한 메모리 자원을 추상화하여 이를 사용하는 사용자들에게 매우 큰 메모리로 보이게 만드는 것



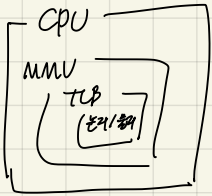
▲ 그림 3-11 가상 메모리



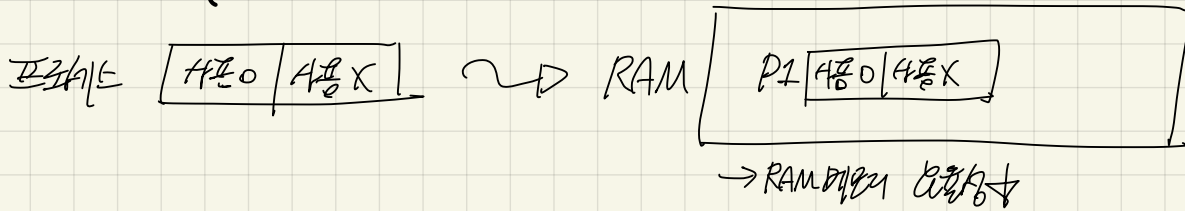
배경 지식



TLB (논리주소와 물리주소의 변환 속도를 높이기 위해 사용하는 캐시)



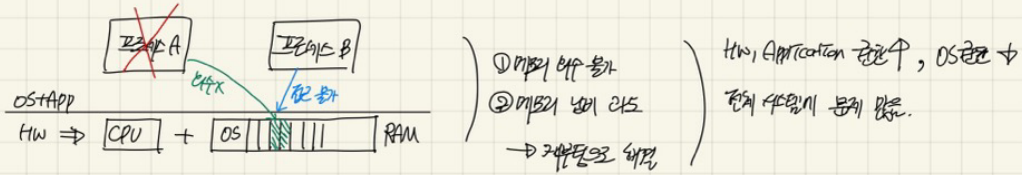
가상 Memory



가상 Memory - Paging

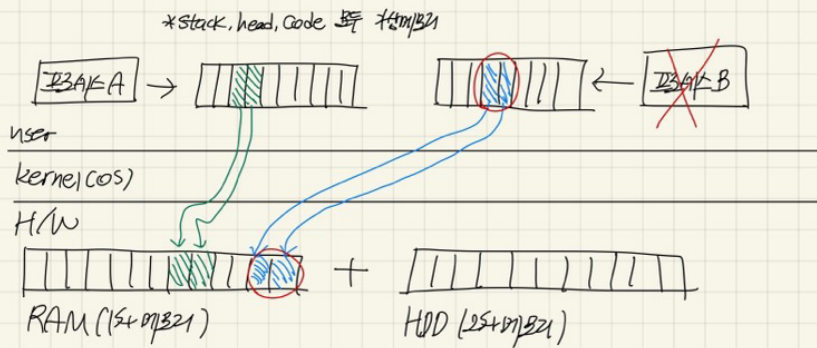
가상메모리

step1) "MS-DOS" OS 사용



step2) 보노드 합 (가상메모리 사용)
+ 가상메모리 사용

가상메모리 구조



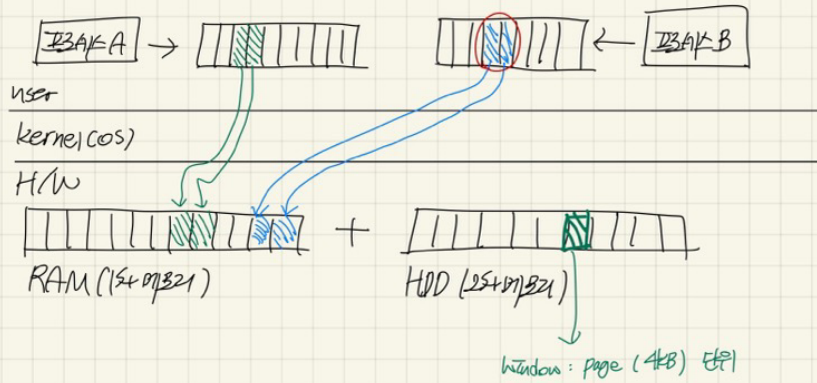
- * stack, heap, code 등 가상메모리
- ① 프로세스 B dead
- ② 가상메모리 실제 메모리 단위 → OS는 알고리즘
- ③ OS가 RAM의 메모리 사용 가능하도록 할 것 (= 바이트)
프로세스 B가 사용하던
- ④ 프로세스 죽/일 OS는 동작X

⇒ 가상 Memory의 사용 이치

* 가상메모리 사용의 미)

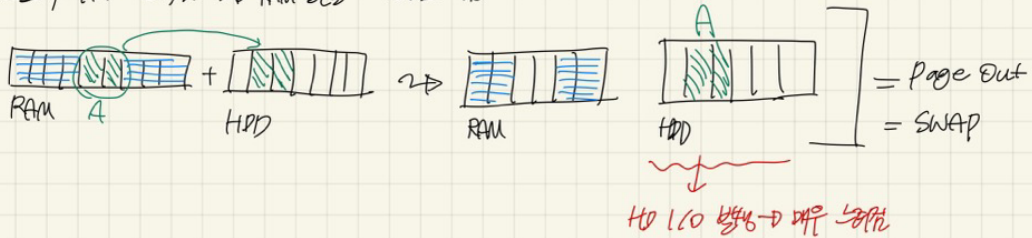
아플 phone → 프로세스 동시에 실행 + 근간에 중지 ... 동작 시도
하루 24시간

가상 메모리 활용



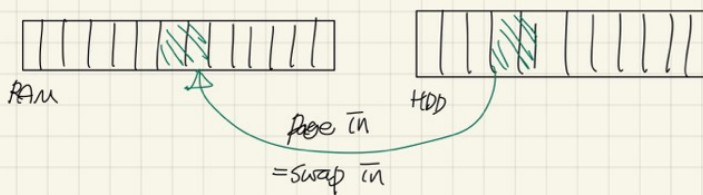
① 메모리 부족 상황 발생

② 프로세스 A가 동작 X → RAM 공간 HDD로 이동



③ RAM의 공간이 부족을 방지

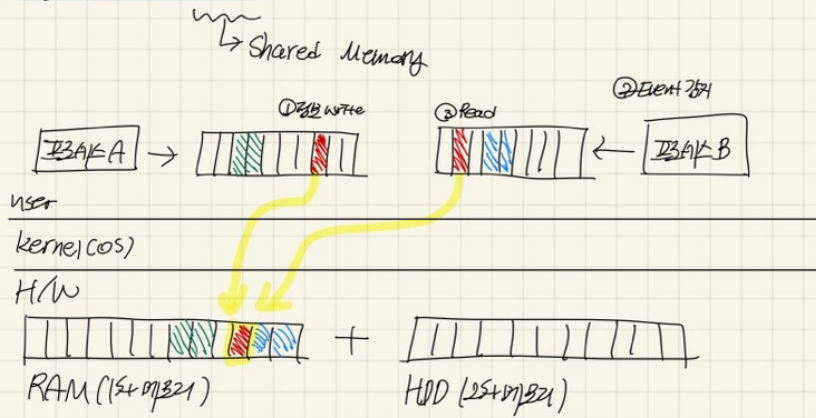
(이동된 프로세스 A 재사용의 상황)



" 주기억장치 (RAM)과 보조저장장치 동시에 하나의 논리적 메모리를 구성하여 사용할 수 있다. "

→ 현재 사용하는 메모리는 OS가 관리 → 메모리의 용량만 늘릴 수 있음.

가상메모리 IPC



* 가상메모리는 독립적 (침범 불가)

→ 의도적으로 공유 가능

①, ②, ③ ⇒ 프로세스간의 정보를 공유하기 위한 방식이며 의도적으로 침범 mapping

→ 사동으로 통할 가능

ex) "북한 통일" 가능

가상메모리 관리 (페이징)

연속메모리 할당 → 문제(1) 대부분 안타

문제(2) 물리메모리보다 큰 프로세스 실행 불가.

"실행상태에 있는 프로세스를 일정한 메모리에 쪼개"

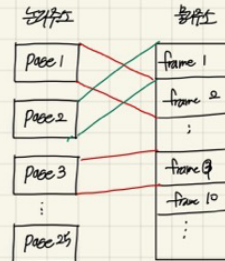
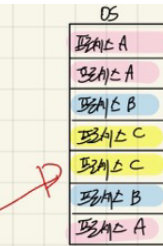
① 페이징이란?

프로세스를 작은 크기로 쪼개 → **물리적으로 작은 단위로 쪼개**

- 논리 주소 공간을 page 단위로 나눔

- 물리 주소 공간을 frame 또는 page에 대한 단위로 나눔

⇒ Page를 frame에 할당하는 가상메모리 관리 방법



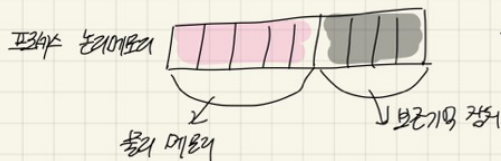
→ 연속된 페이지 X

② 페이징에서의 스왑

- 프로세스 단위 X, Page 단위 스왑 in/out

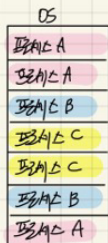
- 메모리에 할당할 필요 없는 페이지들은 물리메모리에서 swap out

- 실행 필요 page는 메모리에서 swap in



⇒ 물리메모리보다 큰 프로세스 실행 가능

③ 페이징 테이블

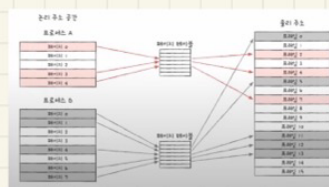
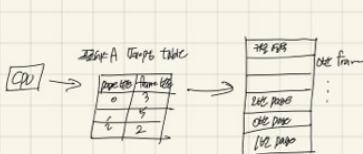


→ 프로세스에 있는 page가 어느 frame에 할당되어 있는지 파악 가능

↓ 페이징 table

= page 번호와 frame 번호를 매핑하는 정보

= 프로세스마다 페이징 table 有



→ CPU 접근하기 어려운 보조기억장치 사용

→ CPU는 논리 주소 해석만 가능

* 페이징 Algorithm : FIFO / LRU / NUR / LFU

6. 프로세스와 스레드

* 빌드 VS 컴파일

① 빌드: 소스 코드 파일을 실행 가능한 실행물로 변환
(컴파일 포함)

② 컴파일: 소스 코드 → binary 코드로 변환

프로세스란?

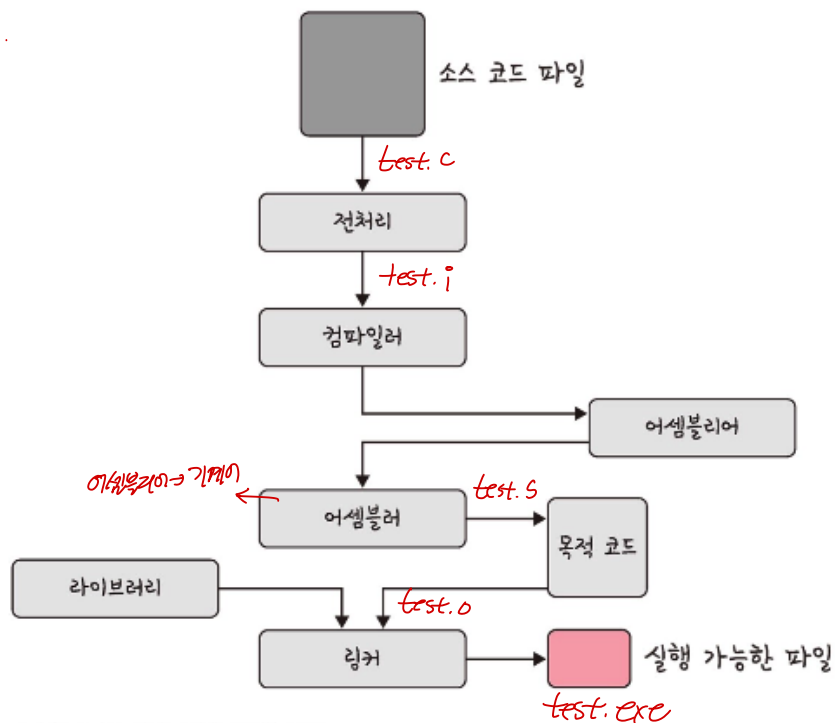
컴퓨터에서 실행되고 있는 프로그램
CPU 스케줄링의 대상이 되는 작업

스레드란?

프로세스 내 작업의 흐름

프로세스와 컴파일 과정

* 컴파일 과정



▲ 그림 3-17 프로그램의 컴파일 과정

* 라이브러리: 여러 다른 목적 코드를 하나로 묶어서 실행할 수 있도록 함

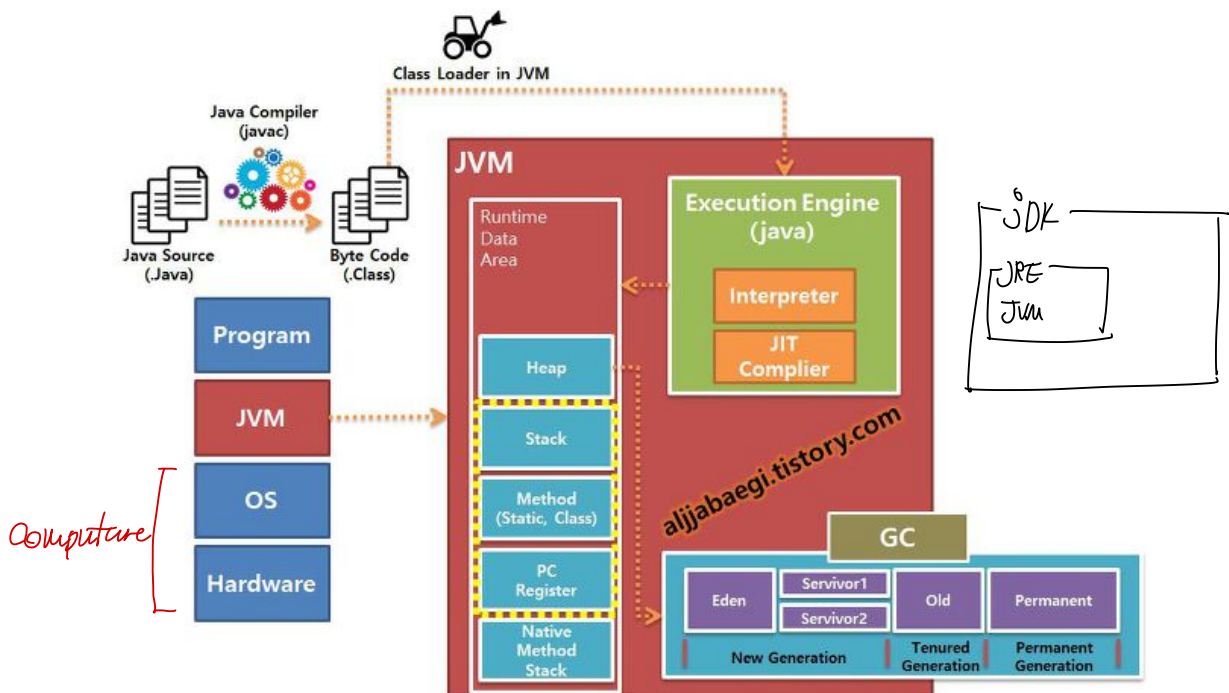
<https://velog.io/@rlvy98/JAVA-자바-컴파일-과정>

JVM

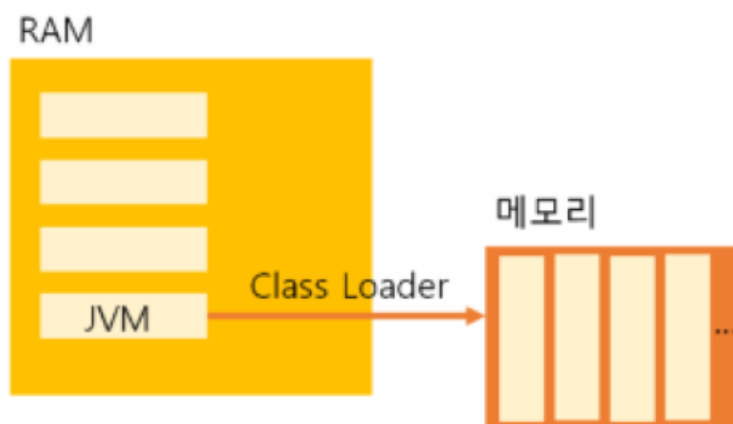
JVM이란?

자바 프로그램 실행환경을 만들어 주는 SW

자바 코드를 컴파일하여 .class 바이트 코드로 만들면 해당 코드는 JVM환경에서 실행
JVM은 자바 실행 환경 JRE에 포함되어 있음



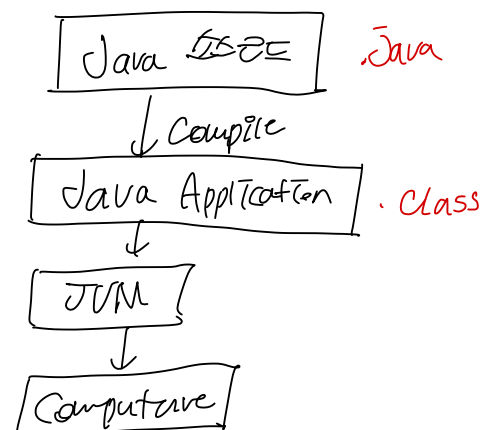
✓ JVM 구조



자바 컴파일러

- 자바 소스코드(.java)를 바이트 코드(.class)로 변환시켜줍니다.

클래스 로더(Class Loader)



- 자바는 동적으로 클래스를 읽어오므로, 프로그램이 실행 중인 런타임에서야 모든 코드가 자바 가상 머신과 연결됩니다. 이렇게 동적으로 클래스를 로딩해주는 역할을 하는 것이 바로 클래스 로더(class loader)입니다.
- 자바에서 소스를 작성하면 .java파일이 생성되고 .java소스를 컴파일러가 컴파일하면 .class파일이 생성되는데 클래스 로더는 .class 파일을 묶어서 JVM이 운영체제로부터 할당받은 메모리 영역인 Runtime Data Area로 적재합니다

실행 엔진(Execution Engine)

JVM으로 로드된 .class 파일(바이트코드)들은 Runtime Data Areas의 Method Area에 배치되는데, 배치된 이후에 JVM은 Method Area의 바이트 코드를 실행 엔진(Execution Engine)에 제공하여, 정의된 내용대로 바이트 코드를 실행시킵니다.

스레드(Thread) 마다 하나씩 생성

- PC 레지스터 : 스레드가 어떤 명령어로 실행되어야 할지 기록하는 부분(JVM 명령의 주소를 보유)
- 스택 Area : 지역변수, 매개변수, 메서드 정보, 임시 데이터 등을 저장
- 네이티브 메서드 스택 : 실제 실행할 수 있는 기계어로 작성된 프로그램을 실행시키는 영역

모든 스레드가 공유해서 사용 (GC의 대상)

- 힙 : 런타임에 동적으로 할당되는 데이터가 저장되는 영역. 객체나 배열 생성이 여기에 해당(또한 힙에 할당된 데이터들은 가비지컬렉터의 대상이 됨. JVM 성능 이슈에서 가장 많이 언급되는 공간)
- 메서드 영역 : JVM이 시작될 때 생성되고, JVM이 읽은 각각의 클래스와 인터페이스에 대한 런타임 상수 풀, 필드 및 메서드 코드, 정적 변수, 메서드의 바이트 코드 등을 보관

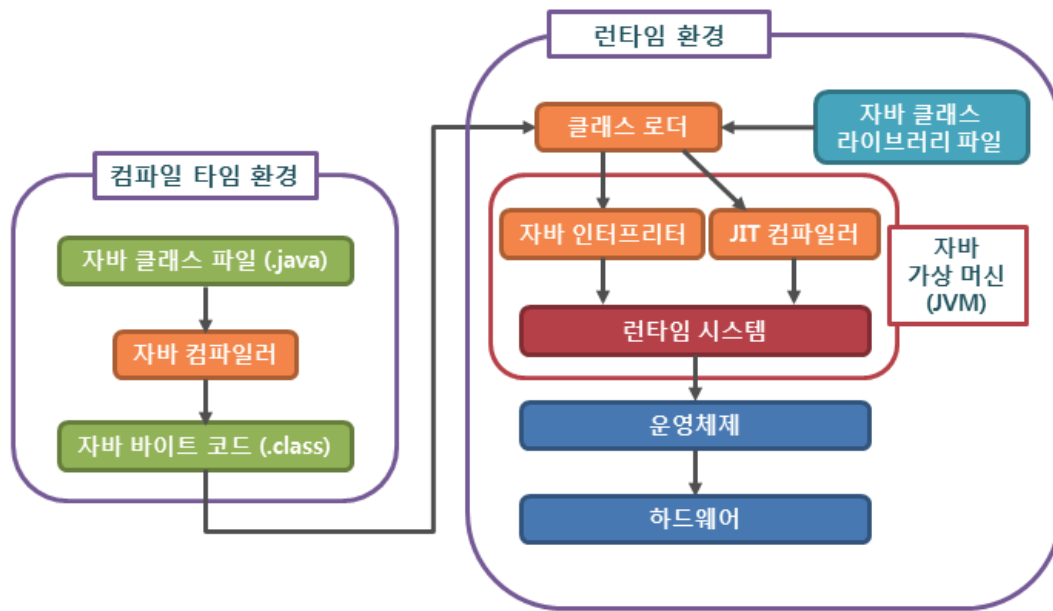
가비지 컬렉터(Garbage Collector)

- 자바 이전에는 프로그래머가 모든 프로그램 메모리를 관리했음 하지만, 자바에서는 JVM이 프로그램 메모리를 관리합니다.
- JVM은 가비지 컬렉션이라는 프로세스를 통해 메모리를 관리함. 가비지 컬렉션은 자바 프로그램에서 사용되지 않는 메모리를 지속적으로 찾아내서 제거하는 역할을 합니다.

실행순서 : 참조되지 않은 객체들을 탐색 후 삭제 → 삭제된 객체의 메모리 반환 → 힙 메모리 재사용

~~java의 컴파일 과정~~

Java 공부



1. 개발자가 자바 소스코드(.java)를 작성

2. 자바 컴파일러(Java Compiler)가 자바 소스파일을 컴파일

이때 나오는 파일은 자바 바이트 코드(.class)파일로 아직 컴퓨터가 읽을 수 없는 자바 가상머신(JVM)이 이해할 수 있는 코드이다.

3. 컴파일된 바이트 코드를 JVM의 클래스로더(Class Loader)에게 전달

4. 클래스 로더는 동적로딩(Dynamic Loading)을 통해 필요한 클래스들을 로딩 및 링크하여 런타임 데이터 영역(Runtime Data Area)

즉, JVM 메모리에 바이트 코드들을 올려줌

5. 실행엔진(Execution Engine)은 JVM 메모리에 올라온 바이트 코드들을 명령어 단위로 하나씩 가져와서 실행합니다.

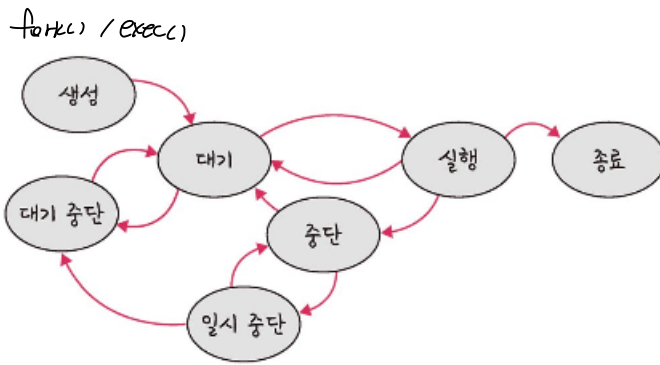
이 때, 실행 엔진은 두 가지 방식이 있다.

5-1) 인터프리터 : 바이트 코드 명령어를 하나씩 읽어서 해석하고 실행한다.

5-2) JIT(Just-In-Time) 컴파일러 : 인터프리터의 단점을 보완하기 위해 도입된 방식, 바이트 코드 전체를 컴파일하여 바이너리 코드로 변경하고 이후에는 해당 메서드를 더 이상 인터프리팅 하지 않고, 바이너리 코드로 직접 실행하는 방식.

(하나씩 인터프리팅하여 실행하는 것이 아니라 바이트 코드 전체가 컴파일된 바이너리 코드를 실행하는 것이기 때문에 실행속도는 더 빠름)

프로세스의 상태



▲ 그림 3-18 프로세스의 상태

프로세스 메모리 구조

컨텍스트 스위칭

7. 멀티프로세싱

- = 동시에 2가지 이상의 일을 수행
- = 1개의 일을 병렬로 처리 가능
- = 특정 프로세스의 메모리, 프로세스 중 일부에 문제 발생 시 다른 프로세스 이용해서 처리 가능

컨테이너 스위칭

= CPU에서 실행 중인 프로세스/스레드와 다른 프로세스/스레드로 교체되는 것

= 여러 프로세스/스레드로 구성된 실행 단위 관리

= OS kernel에서 지원